ASSESSMENT 02

Analysing the Role of AI-Assisted Programming in Modern Software Development

S.M.D.L. Wijerathne

2023s20292

s17541

Table of Contents

Conceptual Understanding

What is AI-Assisted Programming?

AI-assisted programming is the use of artificial intelligence tools to help software developers write, understand, test, and improve computer programs. Instead of writing every line of code manually, developers can use AI tools to generate code, suggest improvements, identify errors, explain complex programming concepts, and automate repetitive tasks.

Popular AI tools such as ChatGPT, GitHub Copilot, Amazon CodeWhisperer, and Google Gemini use machine learning models that have been trained on large amounts of programming knowledge. These tools can understand user instructions written in natural language and generate code in different programming languages.

AI-assisted programming does not replace programmers. Instead, it acts as a supportive assistant that helps developers work more efficiently while allowing them to focus on problem-solving and software design. However, developers must still review, test, and validate AI-generated code before using it in real applications.

Types of AI Coding Tools

- Code Generation Tools

These tools generate source code based on user instructions. Developers can describe what they need in plain English, and the AI produces the corresponding code.

Examples:

- ChatGPT
- GitHub Copilot
- Amazon CodeWhisperer

- Debugging Tools

Debugging tools help developers identify programming errors, explain why they occur, and suggest possible solutions. They can reduce the time spent finding bugs in large programs.

Examples:

- ChatGPT
- GitHub Copilot Chat

- Code Refactoring Tools

Refactoring tools improve the structure and readability of existing code without changing its functionality. They help developers produce cleaner and more maintainable software.

Examples:

- GitHub Copilot
- JetBrains AI Assistant

- Documentation Generation Tools

These tools automatically generate comments, documentation, and API descriptions from source code. This helps developers maintain well-documented projects.

Examples:

- ChatGPT
- GitHub Copilot

- Test Case Generation Tools

AI tools can create unit tests and test cases automatically, helping developers improve software reliability and reduce manual testing effort.

Examples:

- ChatGPT
- GitHub Copilot

Advantages of AI-Assisted Programming

➢ Faster Development

AI can generate code quickly, allowing developers to complete programming tasks in less time. This increases overall productivity and speeds up software development.

➢ Improved Learning

Students and beginner programmers can use AI tools to understand programming concepts, learn new languages, and receive explanations for coding errors.

➢ Reduced Repetitive Work

Many programming tasks, such as writing boilerplate code, creating getters and setters, and generating documentation, can be completed automatically using AI tools.

➢ Better Code Suggestions

AI provides recommendations for improving code quality, readability, and efficiency. Developers can use these suggestions to produce cleaner and more maintainable programs.

➢ Faster Debugging

AI tools can quickly identify syntax errors, logical mistakes, and potential bugs, making it easier to troubleshoot and fix problems.

Risks and Limitations of AI-Assisted Programming

1. Incorrect or Hallucinated Code

AI sometimes generates code that appears correct but contains logical errors or does not solve the intended problem. Developers must carefully review all AI-generated code.

2. Security Risks

AI-generated code may contain security vulnerabilities if it is used without proper validation. Sensitive applications require careful security testing regardless of how the code was produced.

3. Overdependence on AI

If developers rely too heavily on AI tools, they may not fully understand the code they use. This can reduce problem-solving skills and make it difficult to debug software independently.

4. Limited Understanding of Project Context

AI may not fully understand the requirements, architecture, or business logic of a software project. As a result, generated code may require significant modifications before it can be integrated into the system.

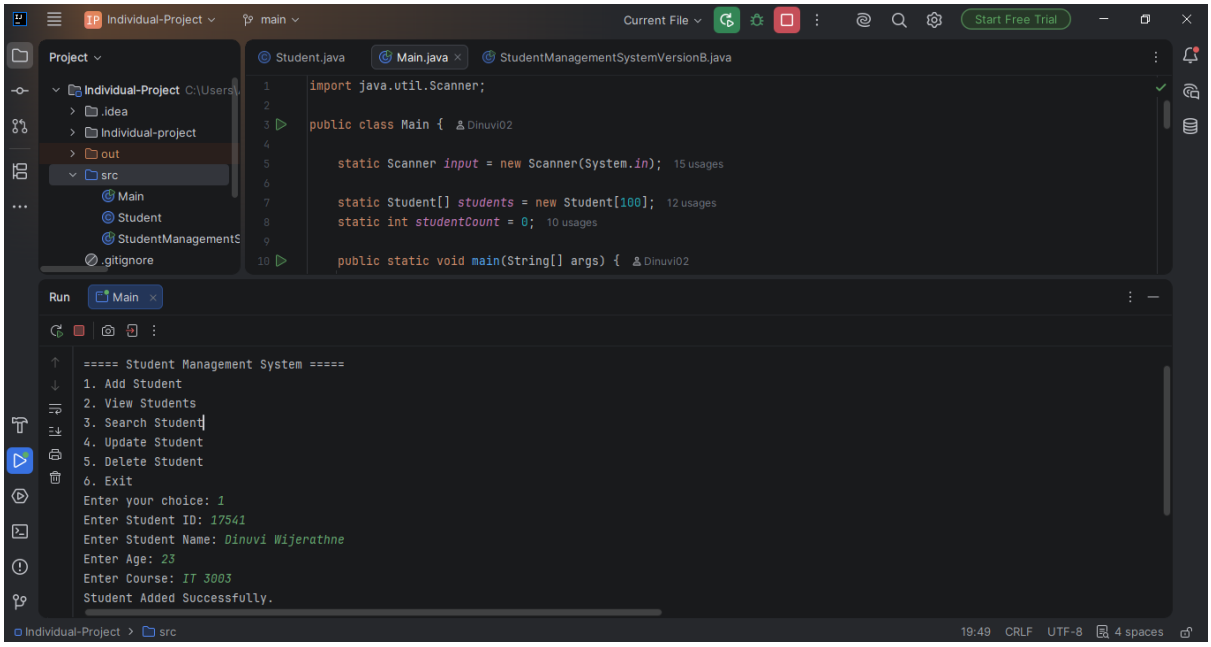
5. Ethical and Copyright Concerns

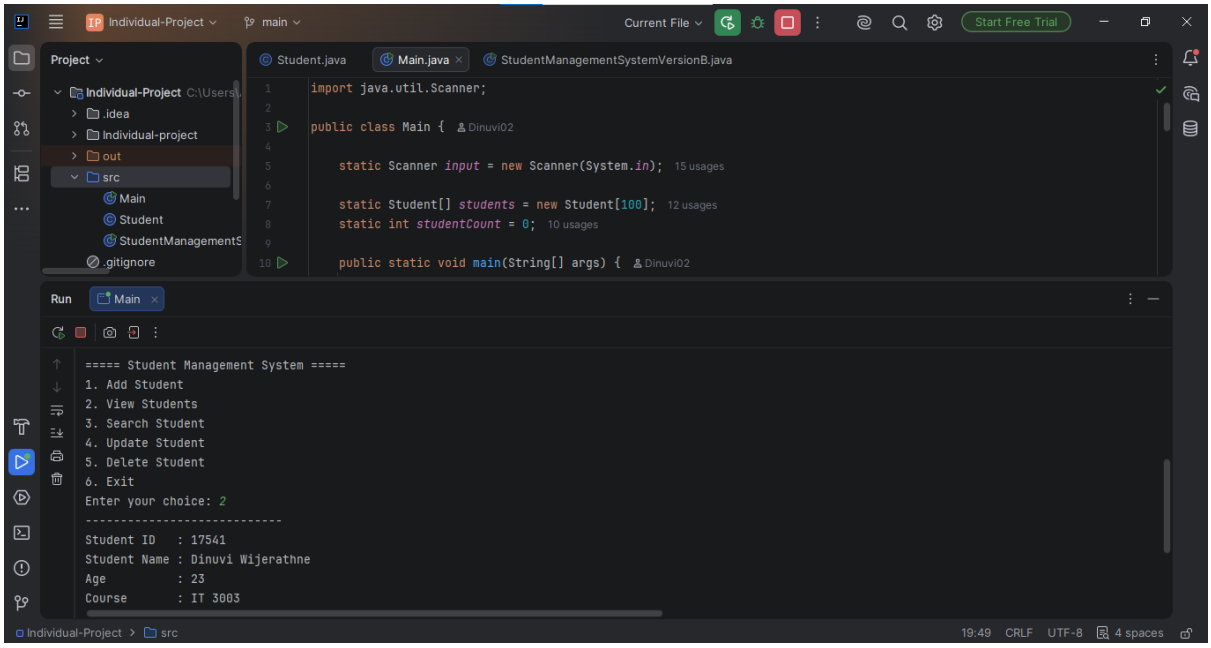
Developers should consider whether AI-generated code respects software licenses and intellectual property rights. They should also acknowledge the use of AI tools when required by academic or professional guidelines

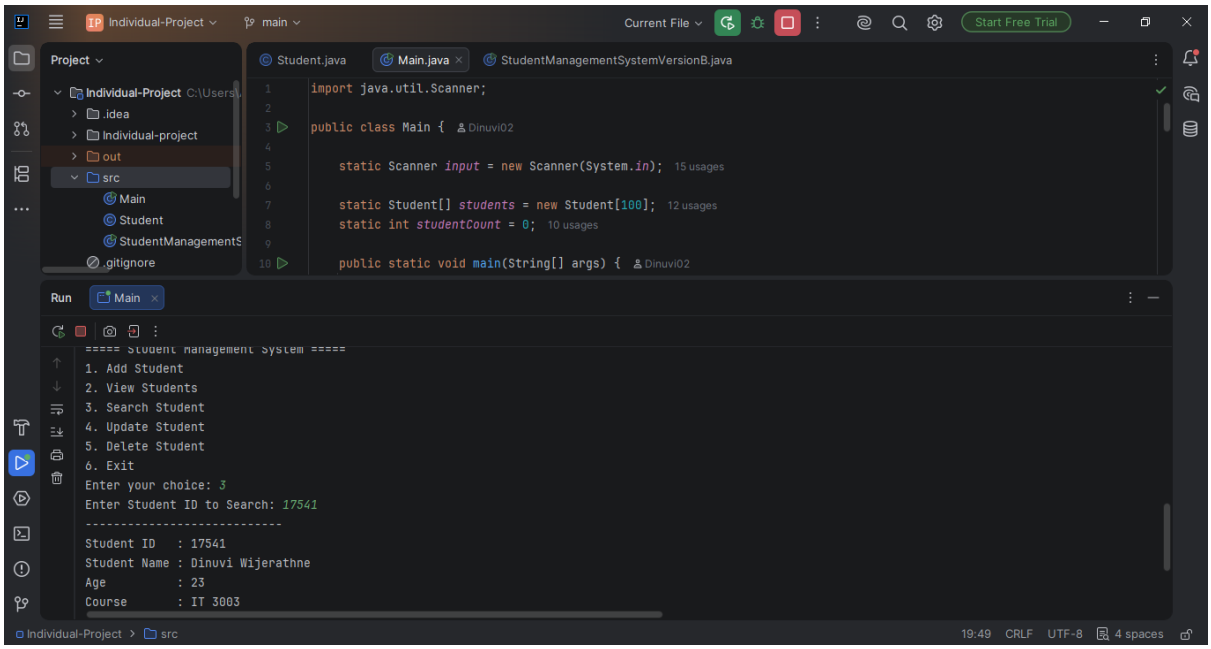
Practical Implementation

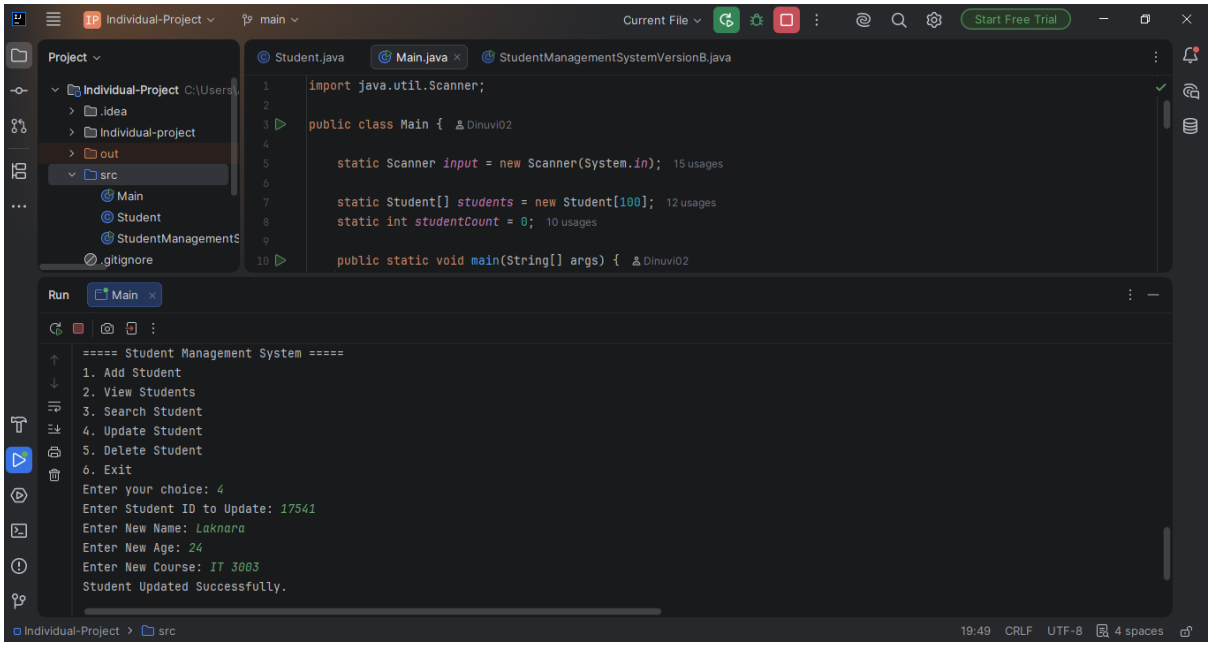
Version A – Traditional Programming Approach

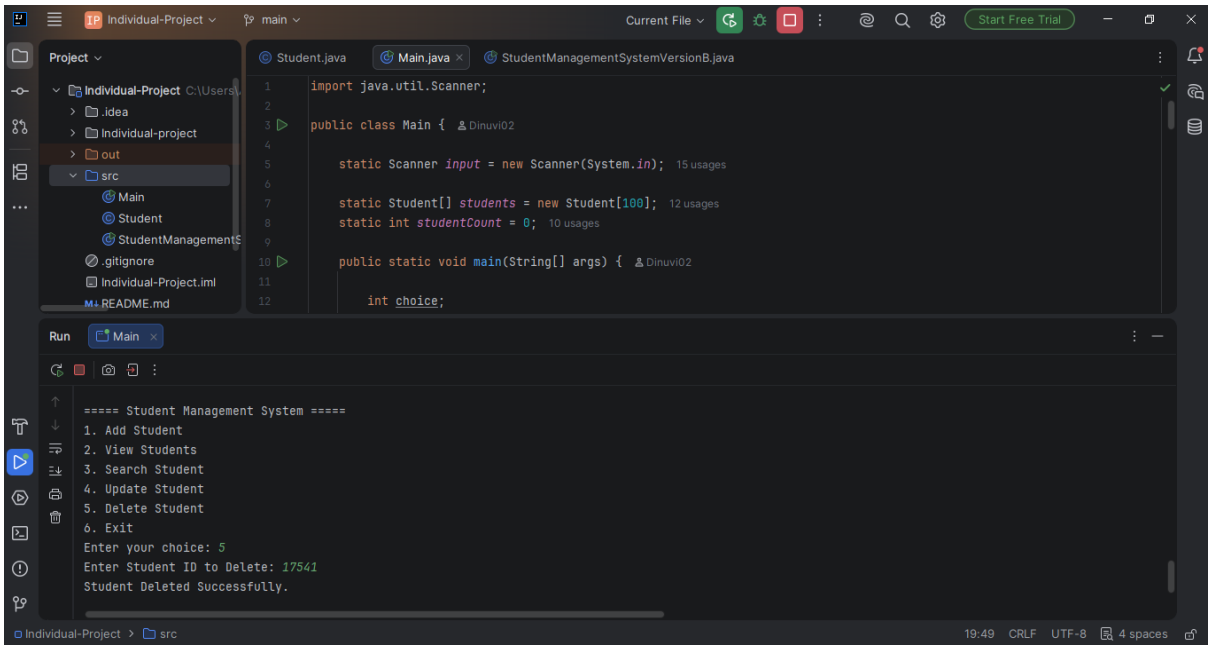
The first version of the Student Management System was developed using the traditional programming approach. In this version, I completed the entire development process manually without using any AI-assisted coding tools. The project was built using Java and Object-Oriented Programming (OOP) concepts to create a simple console-based application for managing student records.




Individual-Project
main
Current File
Start Free Trial
Project
Student.java
Main.java
StudentManagementSystemVersionB.java
import java.util.Scanner;
public class Main {
Dinuvi02
static Scanner input = new Scanner(System.in);
15 usages
static Student[] students = new Student[100];
12 usages
static int studentCount = 0;
10 usages
public static void main(String[] args) {
Run
Main
===== Student Management System =====
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter your choice: 2
Student ID : 17541
Student Name : Dinuvi Wijerathne
Age : 23
Course : IT 3003
19:49
CRLF
UTF-8
4 spaces


Individual-Project
main
Current File
Start Free Trial
Project
Student.java
Main.java
StudentManagementSystemVersionB.java
import java.util.Scanner;
public class Main {
Dinuvi02
static Scanner input = new Scanner(System.in);
15 usages
static Student[] students = new Student[100];
12 usages
static int studentCount = 0;
10 usages
public static void main(String[] args) {
Run
Main
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter your choice: 3
Enter Student ID to Search: 17541
Student ID : 17541
Student Name : Dinuvi Wijerathne
Age : 23
Course : IT 3003
19:49
CRLF
UTF-8
4 spaces





Development Process

The following activities were completed manually during the development process:

- Planned the overall structure of the application.
- Designed the Student Management System based on the project requirements.
- Created all Java classes manually.

- Implemented CRUD (Create, Read, Update, Delete) operations.
- Developed the menu-driven console interface.
- Wrote all methods using Java programming concepts.
- Tested each function individually.
- Fixed syntax errors and logical errors through debugging.

Technologies Used

- Java Programming Language
- Java Collections Framework (ArrayList)
- Object-Oriented Programming (OOP)
- Console-based User Interface

OOP Concepts Applied

The following Object-Oriented Programming concepts were used:

- **Class** – Created classes such as Student, StudentManager, and Main.
- **Object** – Created student objects to store individual student records.
- **Encapsulation** – Declared data members as private and accessed them through getter and setter methods.
- **Constructor** – Used constructors to initialize student objects.
- **Methods** – Implemented methods for adding, searching, updating, displaying, and deleting student records.

Features Implemented

The system provides the following functionalities:

- Add a new student
- View all students
- Search a student by Student ID
- Update student information
- Delete a student record
- Exit the application

Advantages of the Traditional Approach

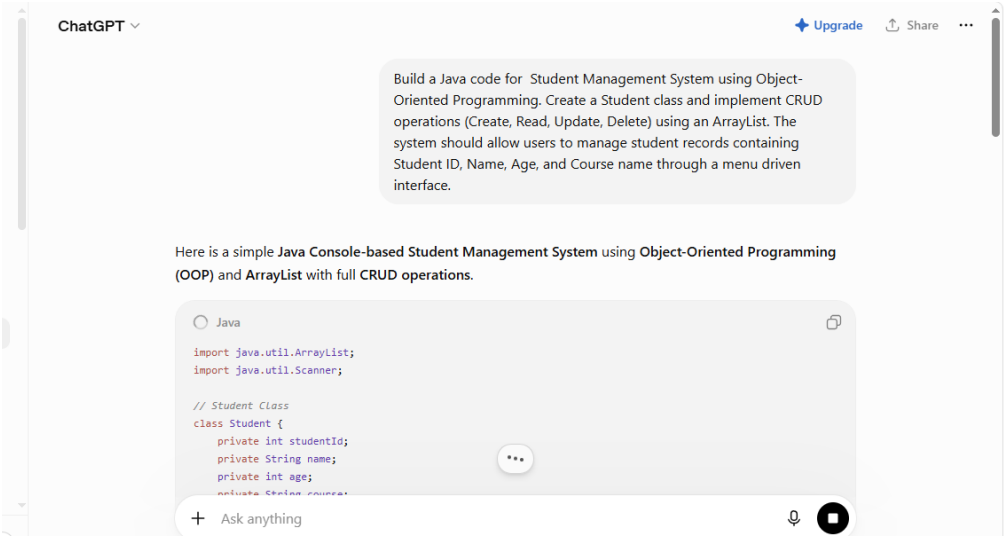
- Improved my understanding of Java programming.
- Enhanced my problem-solving and debugging skills.
- Allowed me to understand the logic behind every method.
- Helped me practice Object-Oriented Programming concepts.

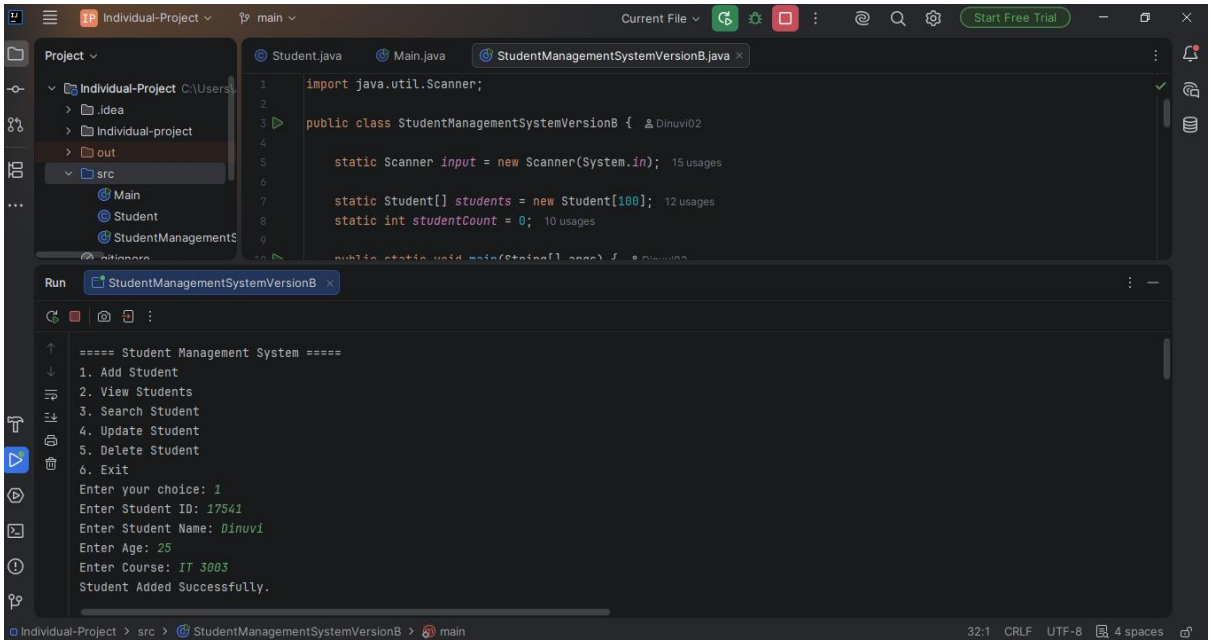
Limitations of the Traditional Approach

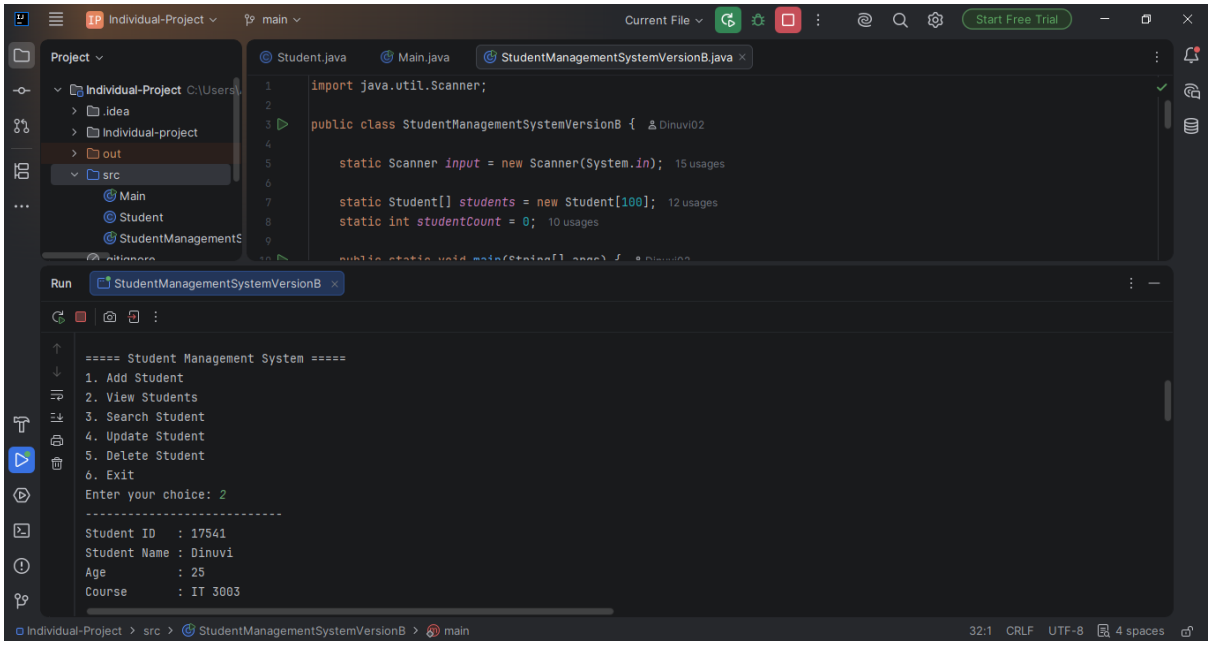
- Required more development time.
- Debugging took longer because all errors had to be identified manually.
- Writing repetitive code reduced productivity.
- Implementing validation and exception handling required additional effort.

Version B – AI-Assisted Programming Approach

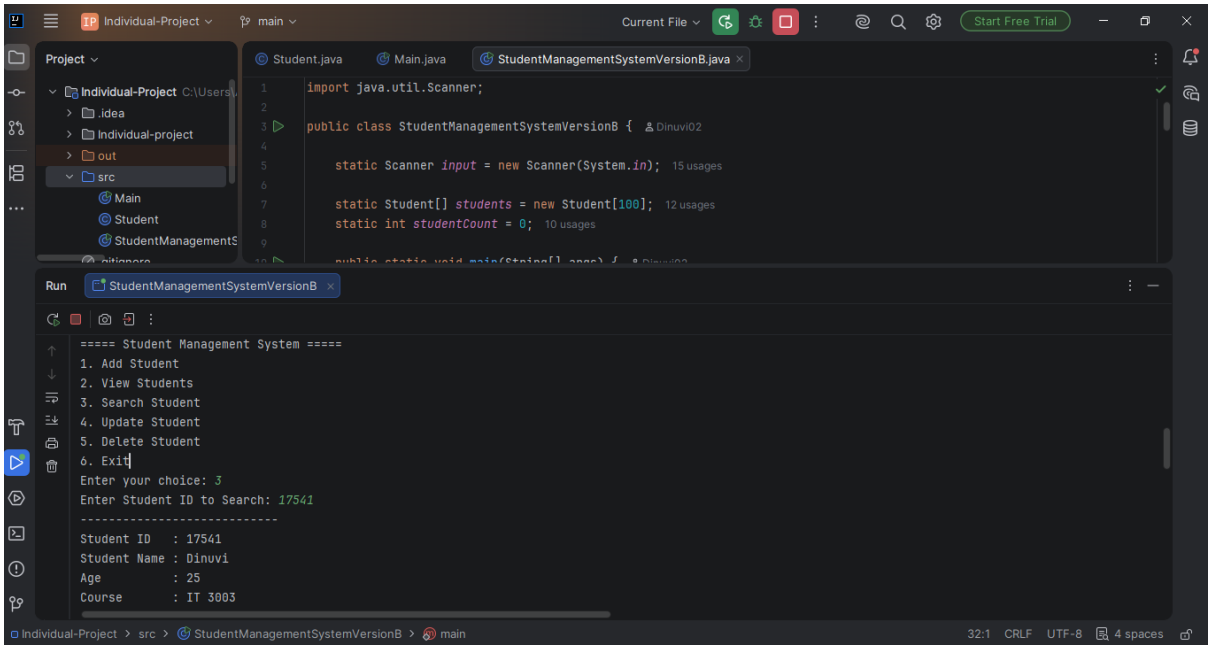
The second version of the Student Management System was developed with the assistance of ChatGPT. AI-assisted programming was used to generate the initial implementation of the Java classes and methods. Instead of writing every line of code manually, I used AI to obtain code suggestions, which I then reviewed, modified, tested, and integrated into the project.

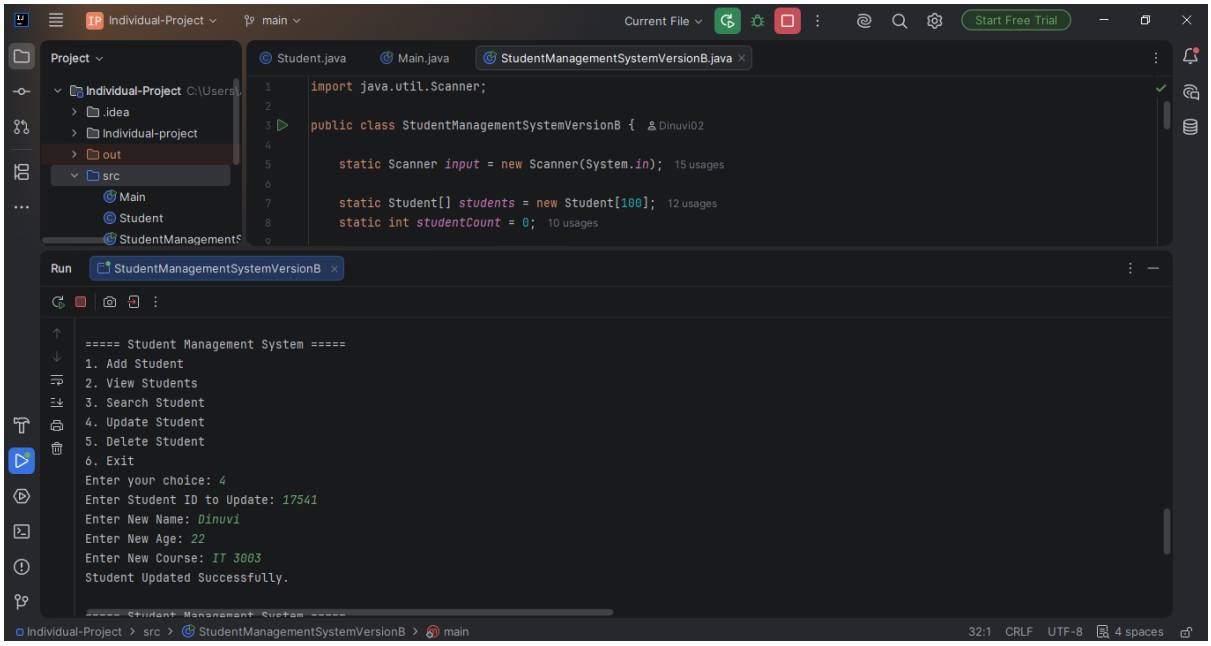


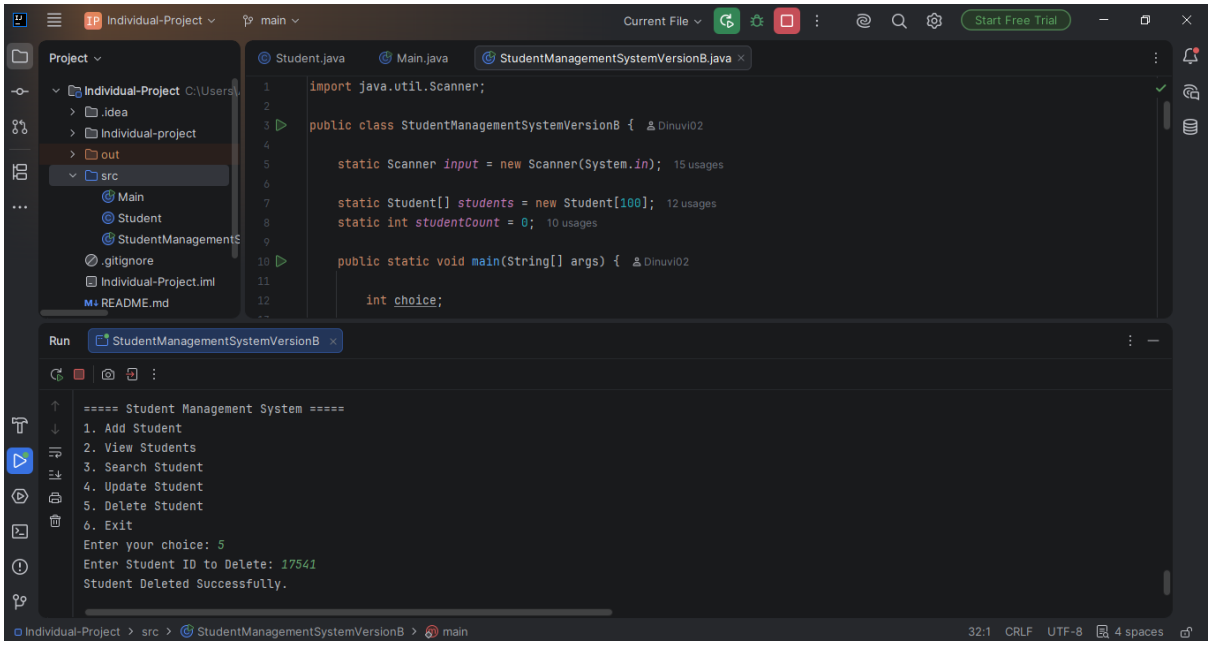

Individual-Project
main
Current File
Start Free Trial
Project
Individual-Project C:\Users\
.idea
Individual-project
out
src
Main
Student
StudentManagementS
Student.java
Main.java
StudentManagementSystemVersionB.java
import java.util.Scanner;
public class StudentManagementSystemVersionB {
Dinuvi02
static Scanner input = new Scanner(System.in);
15 usages
static Student[] students = new Student[100];
12 usages
static int studentCount = 0;
10 usages
Run
StudentManagementSystemVersionB
===== Student Management System =====
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter your choice: 1
Enter Student ID: 17541
Enter Student Name: Dinuvi
Enter Age: 25
Enter Course: IT 3003
Student Added Successfully.
Individual-Project › src › StudentManagementSystemVersionB › main
32:1
CRLF
UTF-8
4 spaces


Individual-Project
main
Current File
Start Free Trial
Project
Individual-Project C:\Users\
.idea
Individual-project
out
src
Main
Student
StudentManagementS
Student.java
Main.java
StudentManagementSystemVersionB.java
import java.util.Scanner;
public class StudentManagementSystemVersionB {
Dinuvi02
static Scanner input = new Scanner(System.in);
15 usages
static Student[] students = new Student[100];
12 usages
static int studentCount = 0;
10 usages
Run
StudentManagementSystemVersionB
===== Student Management System =====
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter your choice: 2

Student ID : 17541
Student Name : Dinuvi
Age : 25
Course : IT 3003
Individual-Project › src › StudentManagementSystemVersionB › main
32:1
CRLF
UTF-8
4 spaces


Individual-Project
main
Current File
Start Free Trial
Project
Individual-Project C:\Users\
.idea
Individual-project
out
src
Main
Student
StudentManagementS
Student.java
Main.java
StudentManagementSystemVersionB.java
import java.util.Scanner;
public class StudentManagementSystemVersionB {
Dinuvi02
static Scanner input = new Scanner(System.in);
15 usages
static Student[] students = new Student[100];
12 usages
static int studentCount = 0;
10 usages
Run
StudentManagementSystemVersionB
===== Student Management System =====
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter your choice: 3
Enter Student ID to Search: 17541
Student ID : 17541
Student Name : Dinuvi
Age : 25
Course : IT 3003
Individual-Project › src › StudentManagementSystemVersionB › main
32:1
CRLF
UTF-8
4 spaces


Individual-Project
main
Current File
Start Free Trial
Project
Individual-Project C:\Users\
.idea
Individual-project
out
src
Main
Student
StudentManagementS
Student.java
Main.java
StudentManagementSystemVersionB.java
import java.util.Scanner;
public class StudentManagementSystemVersionB {
Dinuvi02
static Scanner input = new Scanner(System.in);
15 usages
static Student[] students = new Student[100];
12 usages
static int studentCount = 0;
10 usages
Run
StudentManagementSystemVersionB
===== Student Management System =====
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter your choice: 4
Enter Student ID to Update: 17541
Enter New Name: Dinuvi
Enter New Age: 22
Enter New Course: IT 3003
Student Updated Successfully.
Individual-Project › src › StudentManagementSystemVersionB › main
32:1
CRLF
UTF-8
4 spaces



AI Tool Used

- ChatGPT

AI Prompts Used During Development

The following prompts were used while developing the project:

1. Create a Java Student class using encapsulation.
2. Generate CRUD operations using ArrayList.
3. Create a menu-driven Java Student Management System.
4. Improve input validation and exception handling.
5. Suggest improvements for code readability and maintainability.

Development Process

The following steps were completed while using AI assistance:

- Generated the initial Java classes using ChatGPT.
- Used AI-generated CRUD methods as a starting point.
- Reviewed the generated code carefully.
- Modified variable names and method implementations where necessary.
- Added additional validation and improved program flow.

- Tested every function before including it in the final application.
- Corrected logical and compilation errors identified during testing.

Technologies Used

- Java Programming Language
- ChatGPT
- Java Collections Framework (ArrayList)
- Object-Oriented Programming (OOP)

Features Implemented

The AI-assisted version includes the following features:

- Add Student
- View Students
- Search Student
- Update Student
- Delete Student
- Input validation
- Exception handling
- Improved menu interface
- Better output formatting

Advantages of AI-Assisted Programming

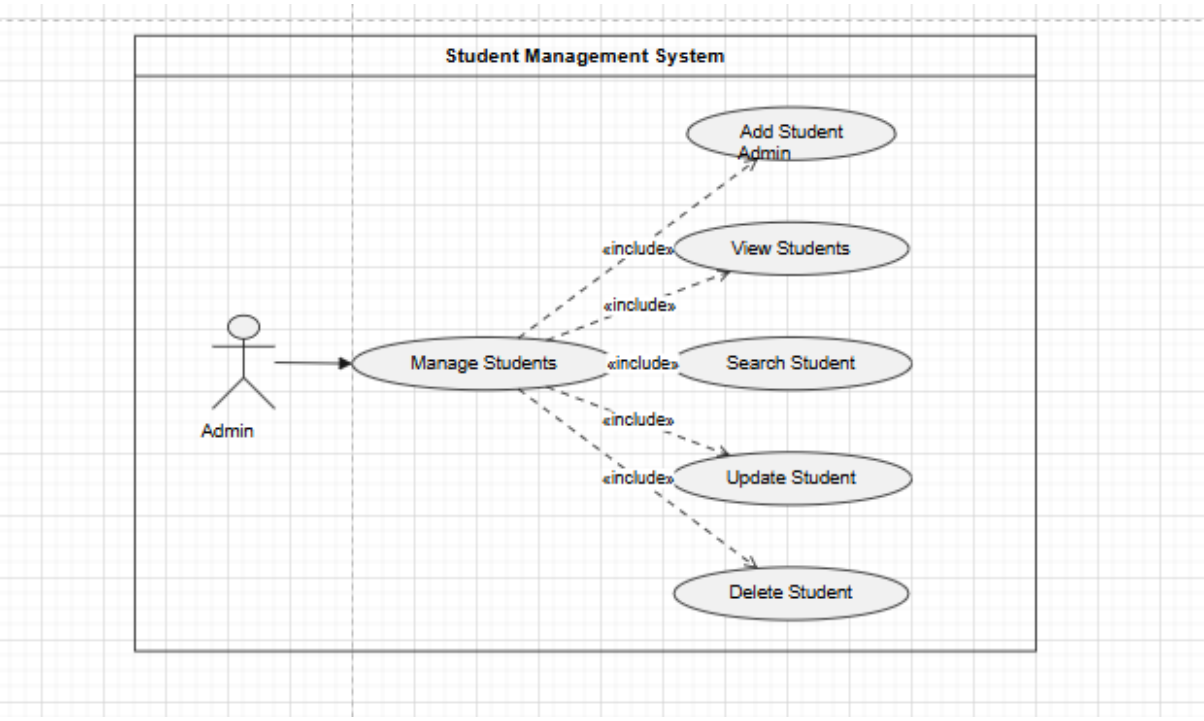
- Reduced the overall development time.
- Generated the initial code structure quickly.
- Improved code readability.
- Suggested better programming practices.
- Helped identify coding mistakes more easily.
- Assisted in improving exception handling and validation.

Limitations of AI-Assisted Programming

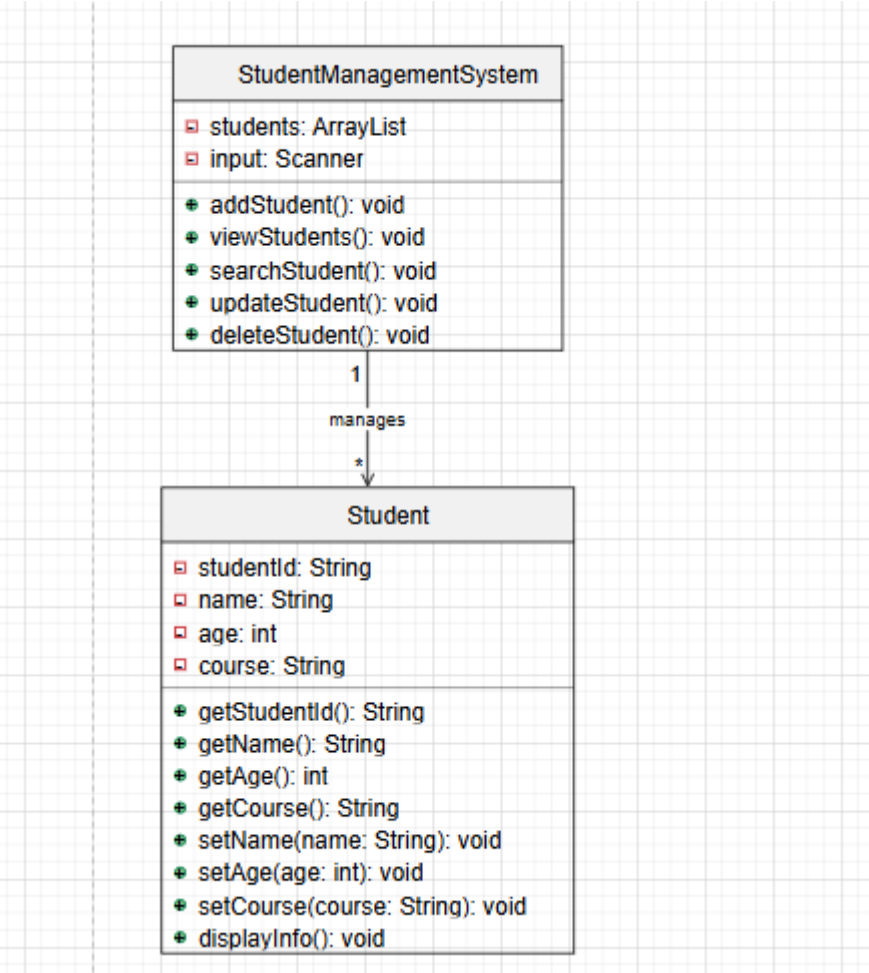
- Some generated code required modification before use.

- AI occasionally suggested unnecessary or overly complex code.
- The generated code had to be reviewed carefully to ensure correctness.
- AI did not always understand the specific project requirements.
- Developers still need programming knowledge to verify and test AI-generated code.

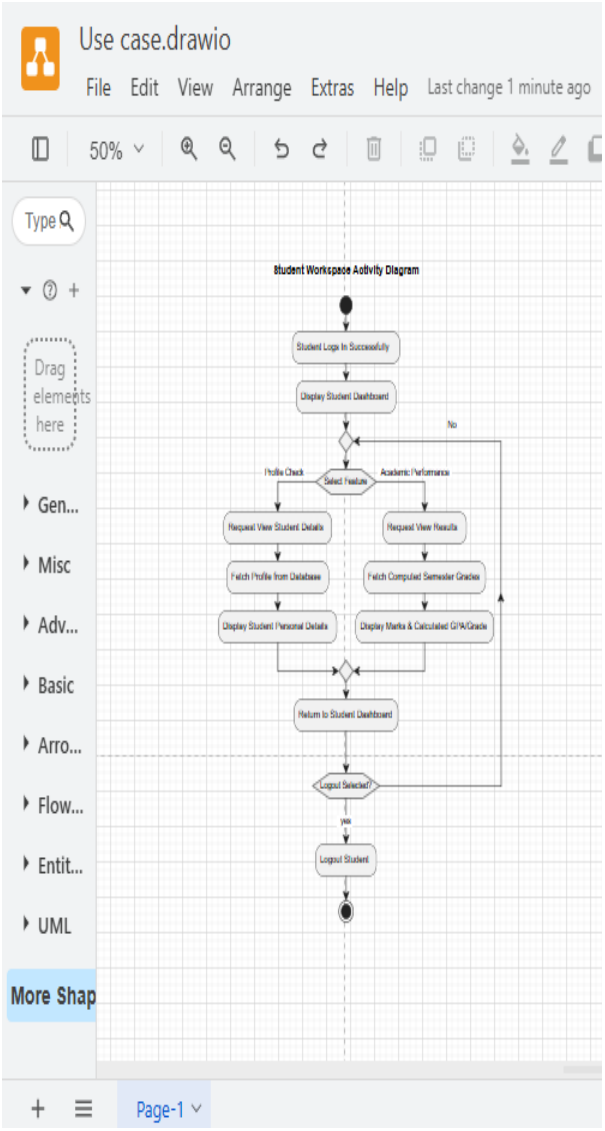
Use Case Diagram



Class Diagram



Activity Diagram



Critical Analysis

How AI Changed My Coding Approach

Developing the Student Management System using both traditional programming and AI-assisted programming allowed me to compare the two approaches. I found that AI significantly changed my development process by making coding faster and providing useful suggestions. However, I still needed to understand and verify the generated code before using it.

How AI Changed My Approach

AI influenced my coding approach in several ways:

- Faster Development
    - AI generated the basic structure of classes and methods within a short time.
    - This reduced the time spent writing repetitive code.
- Better Code Suggestions
    - AI suggested different ways to implement methods.
    - It provided cleaner and more organized code structures.
- Improved Debugging
    - AI helped identify syntax and logical errors.
    - It suggested possible solutions to fix programming issues.
- Learning New Techniques
    - AI introduced alternative programming methods that I had not used before.
    - It explained Java concepts whenever I asked questions.
- Focus on Problem Solving
    - Since AI handled repetitive coding tasks, I was able to spend more time understanding the system requirements and improving the application.

My Observation

Although AI made development faster, I still needed to review, test, and modify the generated code to ensure that it met the project requirements.

Did AI Improve or Reduce My Understanding?

Using AI during software development affected my learning experience in both positive and negative ways. I realized that AI can improve understanding when used correctly, but excessive dependence may reduce programming skills.

How AI Improved My Understanding

- Helped me understand Object-Oriented Programming concepts.
- Explained Java syntax and programming logic clearly.
- Provided examples for methods and classes.
- Assisted in understanding CRUD operations.
- Helped me learn better coding practices.
- Improved my debugging skills by explaining programming errors.

How AI Could Reduce Understanding

- It is easy to copy code without understanding how it works.
- Overdependence on AI can reduce problem-solving ability.
- Students may become less confident when writing programs independently.
- Learning opportunities may decrease if AI is used for every programming task.

My Opinion

In my opinion, AI improves learning only when it is used as a guide. Students should first understand the generated code, test it carefully, and make necessary modifications instead of simply copying it.

Cases Where AI Should Not Be Used

Although AI is a powerful programming assistant, there are situations where developers should avoid depending entirely on AI.

Situations Where AI Should Not Be Used

- During Practical Examinations
    - Students should demonstrate their own programming knowledge without AI assistance.
- Security-Critical Software
    - Applications involving banking, healthcare, or cybersecurity require careful manual review because AI-generated code may contain security vulnerabilities.
- Confidential Projects
    - Sensitive company or customer information should not be shared with AI tools due to privacy concerns.
- When the Developer Does Not Understand the Code
    - AI-generated code should never be used without understanding its functionality.
- Complex Business Logic
    - Projects with unique requirements often require human decision-making and cannot rely entirely on AI-generated solutions.
- Unverified Code
    - Code that has not been tested or reviewed should not be directly implemented in production systems.

My Opinion

AI should be used as a supporting tool rather than replacing the developer's own knowledge and experience.

Ethical Considerations in AI-Generated Code

The increasing use of AI in software development creates several ethical responsibilities for developers. AI-generated code should always be used responsibly and transparently

- *Academic Integrity*

- Students should clearly acknowledge when AI tools have been used.
- AI-generated work should not be presented as completely original work.

- *Code Reliability*

- AI-generated code may contain logical or programming errors.
- Developers are responsible for testing and validating all generated code.

- *Security*

- AI may generate code with security weaknesses.
- Security testing should always be performed before deployment.

- *Copyright and Licensing*

- Developers should ensure that AI-generated code does not violate software licenses or intellectual property rights.

- *Professional Responsibility*

- Developers remain responsible for the final software product.
- AI should support human decision-making rather than replace it.

My Opinion

I believe developers should use AI responsibly by reviewing generated code, testing every feature, and clearly acknowledging the use of AI tools whenever required.

Conclusion

This assignment provided me with valuable experience in developing a **Student Management System** using both the traditional programming approach and the AI-assisted programming approach. By implementing the same project using these two methods, I was able to compare their strengths, limitations, and overall impact on the software development process.

The traditional programming approach helped me strengthen my understanding of Java programming and Object-Oriented Programming (OOP) concepts. Writing the code manually improved my logical thinking, problem-solving ability, debugging skills, and confidence in developing software independently. Although this approach required more time and effort, it gave me a deeper understanding of how each part of the application works.

On the other hand, the AI-assisted programming approach significantly reduced development time by generating the initial structure of the program and providing useful coding suggestions. ChatGPT assisted in creating classes, CRUD operations, improving code readability, and suggesting better programming practices. However, I learned that AI-generated code should never be used without careful review, testing, and modification because it may contain logical errors or may not fully satisfy the project requirements.

GitHub repository

https://github.com/Dinuvi02/IT-3003.git